TUTORIAL SESSIONS

5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 01:
ASCII Invaders Recap

School of Computer Science and Mathematics

LIVERPOOL JOHN MOORES UNIVERSITY

Tutorial Overview

This tutorial is recap tutorial – as this is your first session back, we want to see how much C++ you can remember from the first year of study.

Therefore, we will be recapping on the key concepts and syntax that you learnt in Programming for Games.

We will be creating a space invaders style game, step by step, using Microsoft Visual Studio 2022 and the C++ programming language. In this tutorial, you will be completing the following tasks:

- The **First Task** is to obtain and open the visual studio project for ASCII Invaders.
- The **Second Task** will show you how to compile and run you game.
- The **Third Task** will allow you to control the player ship using keyboard inputs.
- The **Fourth Task** will create an army of enemies to shoot.
- The **Fifth Task** will allow you to fire missiles at the enemies.
- The **Sixth Task** will make the enemies move so they are more difficult to hit.
- The **Seventh Task** will make the enemies shoot back at you!
- The **Eighth Task** will update the score, so you know how well you are doing.
- The **Ninth Task** will check if you have won or lost the game and display a win / lose screen.

Once you have completed the tasks, there are some additional tasks for you to attempt – this will help you recapture your problem-solving skills as well as practicing some debugging and code comprehension.

Task 1: Download and open ASCII Invaders Project

Step 1: Download the ASCII Invaders Project

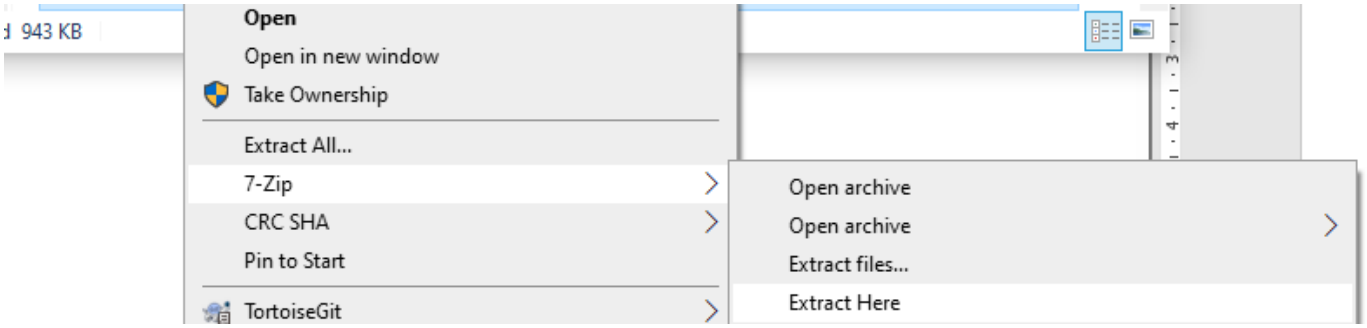
Download the ASCII Invaders Starter zip file (**SEG_CPP_Recap.zip**) from Canvas and save it to your local area of the C drive on the laboratory machines (e.g. C:\users\<username>\, where <username> is your university ID).

It is suggested that you create a folder called **Local.** These files will stay active on the lab machine you are logged into providing you do not exceed 14 days without logging in next.

Step 2: Extract the ASCII Invaders Project

To open the project in Visual Studio, you must extract the project files from the zip file that you downloaded.

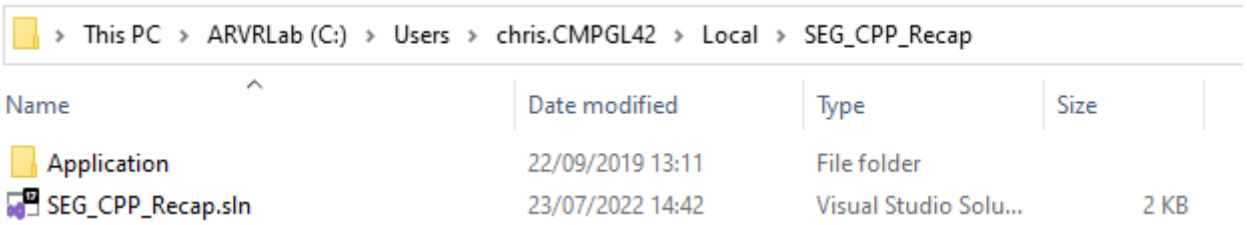
To do this, right click on the **SEG_CPP_Recap.zip** file and select "**7-Zip -> Extract Here**" as shown in the image below:



7-Zip is the recommended method for extracting all zip files you download from Canvas – using the Windows built in Zip file manager causes problems related to the fact that the files you are working within originate from the internet (e.g., downloaded from Canvas).

7-Zip automatically handles these issues when the files are extracted.

Open the **SEG_CPP_Recap** folder that you have just extracted the files to. The folder should contain the following files:



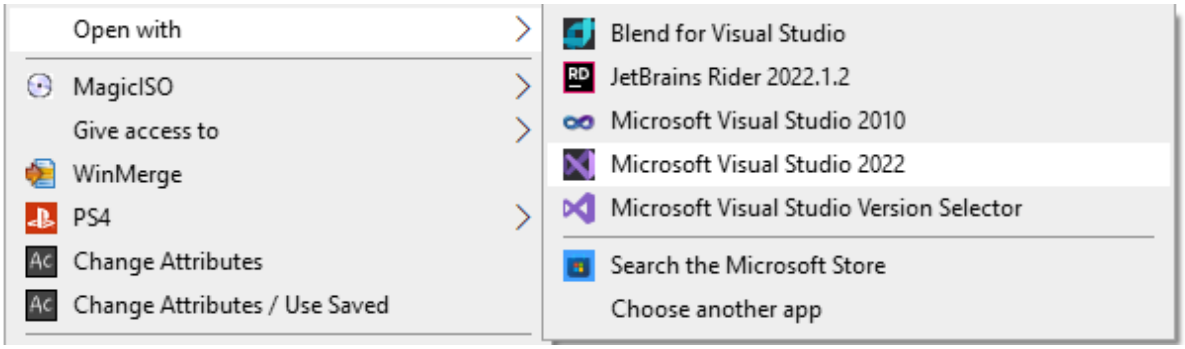
If you do not have these files, then please ask a member of staff for help.

Step 3: Open the ASCII Invaders Visual Studio Project

Now that you have the visual studio project extracted from the zip file, you can open it in Microsoft Visual Studio 2022.

This is the only version of Visual Studio installed in our labs, so you can just double click on the .sln file to open it.

Off campus, if you want to choose the version manually, right click on the **SEG_CPP_Recap.sln** file and select open with -> Microsoft Visual Studio 2022 as shown in the image below:

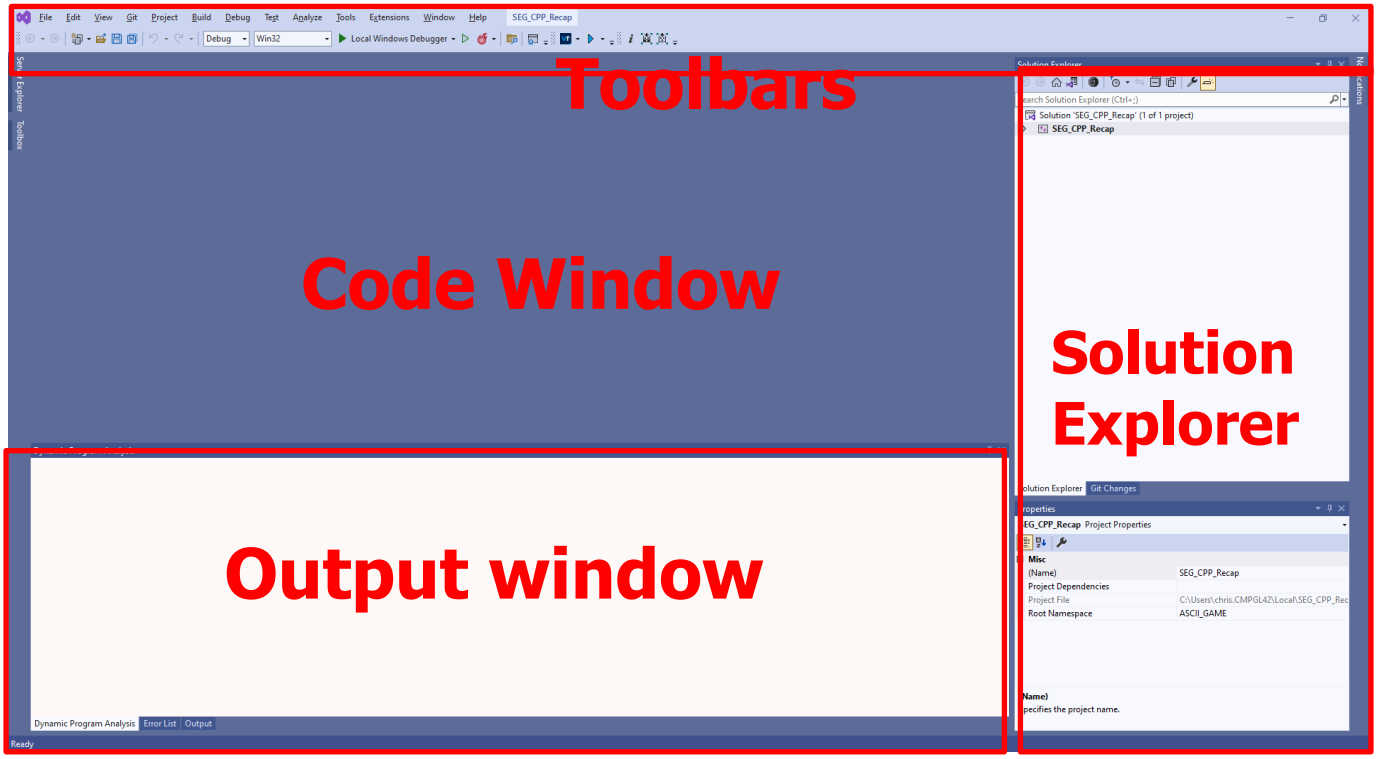


Microsoft Visual Studio 2022 will begin to launch, and you will be presented with the following loading screen:

If this is your first-time logging into your development machine in the Lab, and you have just created your local windows profile account, Visual Studio may take some time to load – it will likely ask you to sign in (*you don't have to but is recommended*).

You may also have licenses to accept, such as the JetBrain's license – it is highly recommended that you do so.

When it has finished loading, you will be presented with a screen that looks like the following image:



Hopefully, you remember all the following from last year.

The Visual Studio interface is split into several sections, as shown above.

- At the top of the screen is the toolbars section. Toolbars contain useful tools to control visual studio. Today we will be using the toolbar to compile and run the ASCII Invaders game.

- On the right of the screen is the solution explorer (it may be the right-hand side if you choose a different development environment when you started Visual Studio). It is in this section that we can see and access all the code files that are contained within the project.

- The code window is in the middle-left of the screen. When we open a code file, it will be displayed here for editing.

We are working with a mini framework which is the same as what you covered last year in your **Programming for Games** module.

Today is all about getting you back up to speed with working in C++, inside Visual Studio, particularly if you have not used it since the end of last semester.

This semester, you will be using Visual Studio's tools more to work as a group and you will also be pair-programming, so it's important that you are comfortable with it – this isn't aiming to be condescending (I promise), I appreciate this will be second nature to a lot of you already!

Task 2: Compiling and Running the game

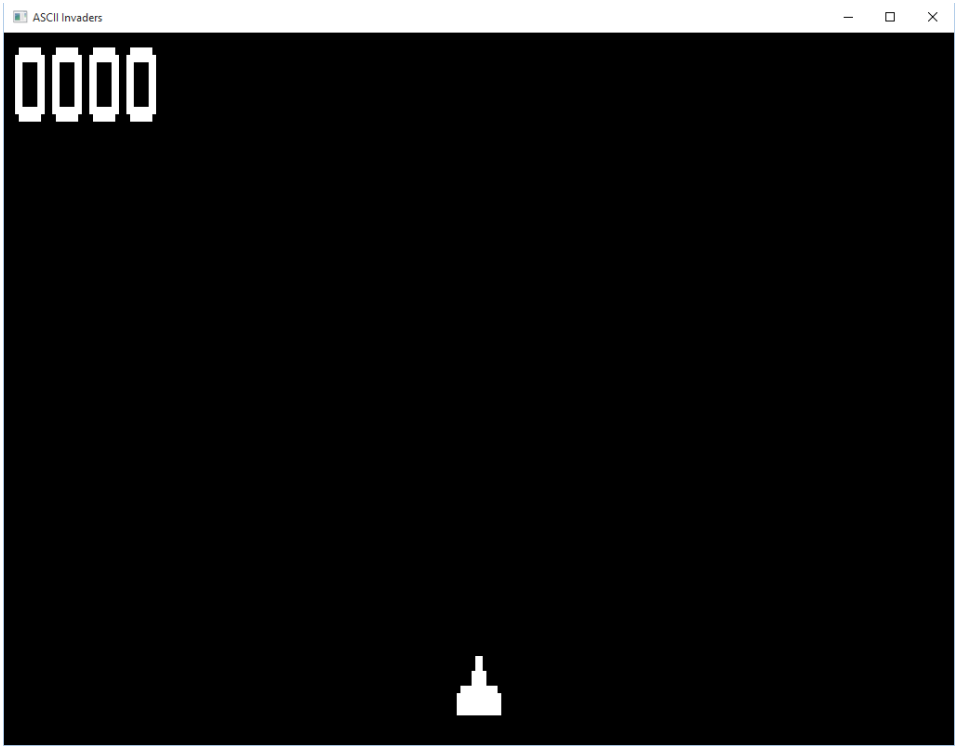
We have obtained and opened the ASCII Invaders project, so now let's compile and run the game to check that it is working.

Step 1: Press the compile and run button

If we want to run the game, we must first compile the code and create an executable. Visual studio provides an easy way of doing this on the toolbar at the top of the screen:

To compile and run the game, click in the compile and run button located on the toolbar at the top of the screen:

The game should compile successfully, an executable will be generated and then automatically run. When the game runs, you will see the following screen appear:



The game currently contains:

- A player ship that cannot move.
- A score that does not function.

What is missing:

- Can’t move the player ship.
- An army of space invaders.
- Can’t fire missiles.
- Can’t score points.
- Can’t win the game.
- Can’t lose the game.

In the subsequent tasks we will add all these features to the game and end up with a space invaders style game!

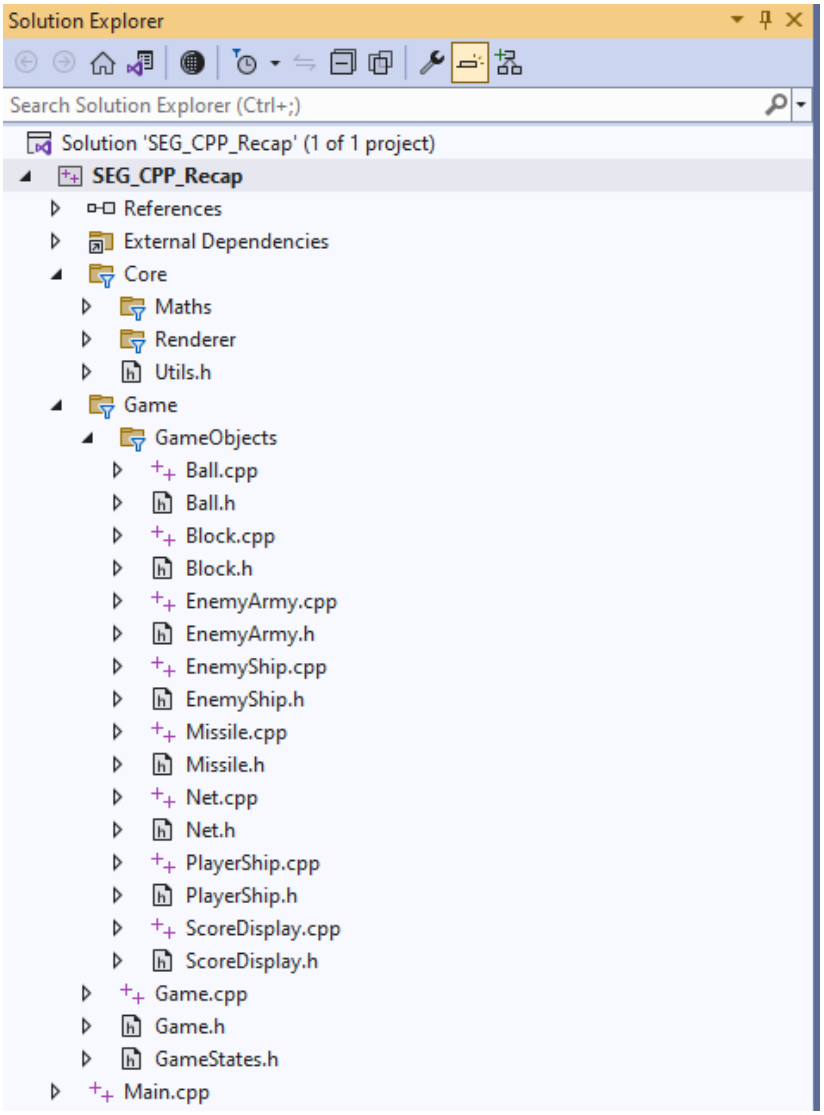
Task 3: Player Ship Movement

Currently the player can't even move their ship. In the original Space Invaders game, the ship could move left and right on the screen. In this task, we will check if the left or right arrow is being pressed on the keyboard and move the ship left or right appropriately.

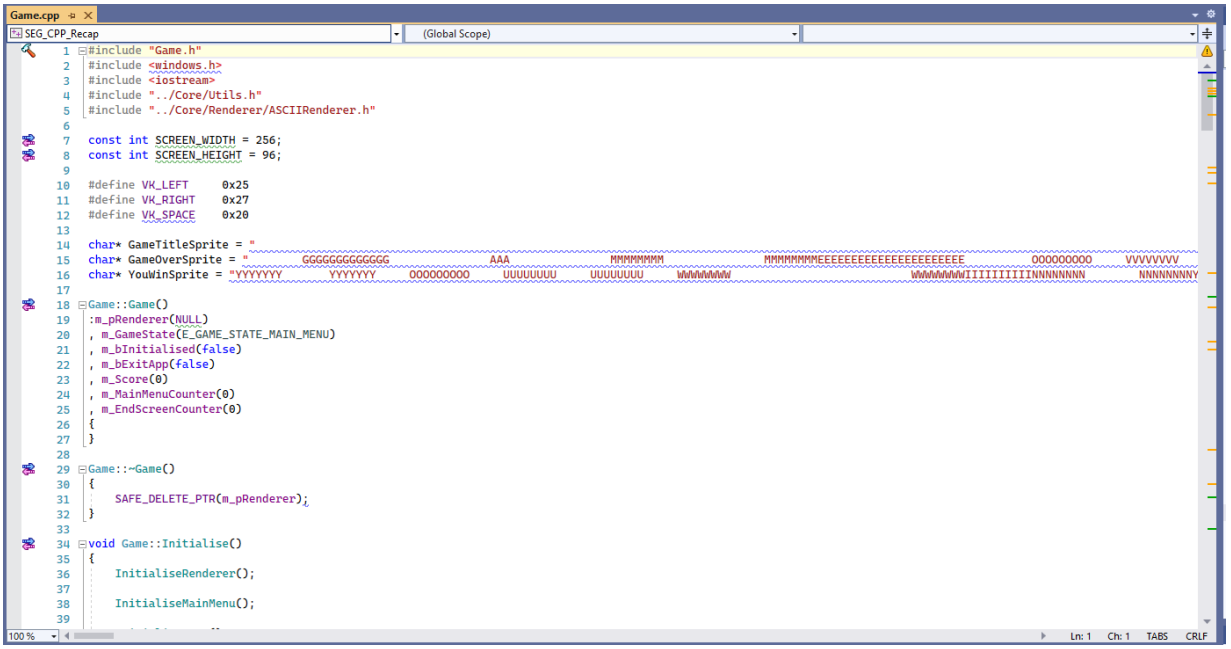
Step 1: Updating the player

For the first step, we need to find where the code is that controls the player ship.

In the solution explorer, find the file called **Game.cpp**.



Double click the Game.cpp file. This should open the Game.cpp file in the code window.



Game.cpp is the main C++ code file where the game logic for ASCII Invaders is written.

Scroll through the **Game.cpp** file to find the UpdateGame() function. It will look like the code below:

```
void Game::UpdateGame()
{

}
```

The UpdateGame() function is where we want to write all of the logic for our game.

In order for our player ship to move, we first want to make sure that we are updating our player ship inside the UpdateGame() function.

We can call the Update() function on the player ship by writing the following line of code:

.

```
    m_PlayerShip.Update();
```

The UpdateGame() function should now look like the following:

```
void Game::UpdateGame()
{
    m_PlayerShip.Update();
}
```

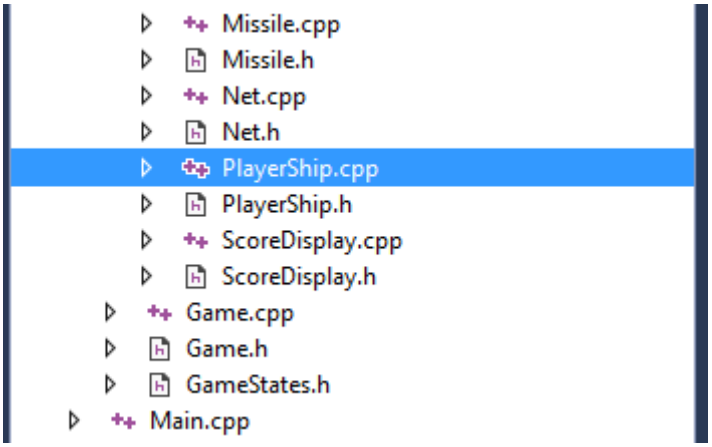
Compile and run the game as you did in Task 2. We are accessing an object called m_PlayerShip. This object is of class type PlayerShip (its datatype). As it is a class instance (an automatic variable) its functions and data that are public are accessible via the . (dot) operator.

If your game didn't run then you will have a compile error!

Check what you have typed matches the code above exactly as in C++ spelling words differently or missing a capital letter will mean the computer will not know what you are asking it to do**. If you cannot get this to work then ask a member of staff for help!**

You will notice that even with this change the player ship still can't move when you press the left and right keys on the keyboard. This is because we need to write some code to check if the keys are being pressed and then move the ship accordingly. As this code is related to moving the player ship then we write the code in the PlayerShip.cpp code file.

Find PlayerShip.cpp in the solution explorer and double click on it to open the file.



We want to put the movement code for the player ship in the Update() function that we called earlier from the Game.cpp code file.

Find the Update() function in the PlayerShip.cpp code file. It should look like the following code:

```
void PlayerShip::Update()
{
    if (!m_bInitialised)
        return;

}
```

Let's move the player to the left when the left arrow key is pressed.

In order to do this we can add the following code to the Update() Function.

```
    if (LeftKeyPressed())
    {
        MoveLeft();
    }
```

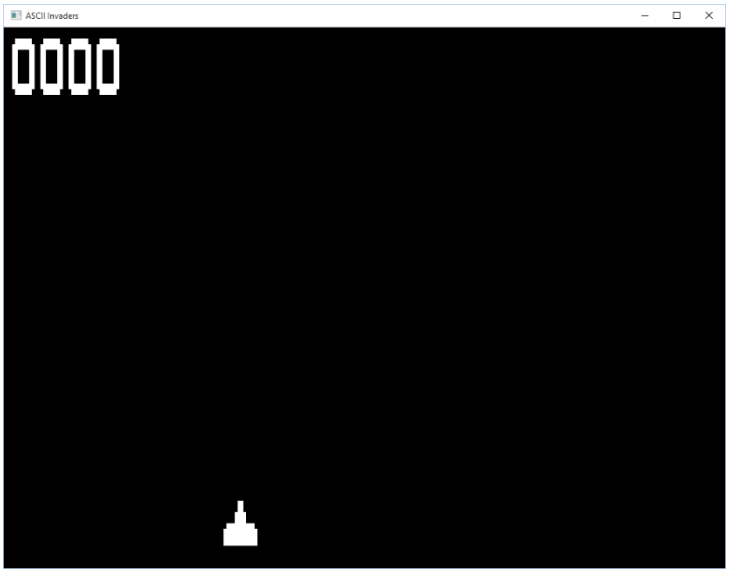
The Update() function should now look like the following

```
void PlayerShip::Update()
{
    if (!m_bInitialised)
        return;

    if (LeftKeyPressed())
    {
        MoveLeft();
    }

}
```

What this code does is checks if the left key is being pressed and if it is it will call the MoveLeft() function. The move left function will take care of moving the player ship for us.

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run then please ask a member of staff for help.**

When the game runs, try pressing the left arrow key on the keyboard. The player ship should now move across the screen to the left as in the image below:



We also need to add code to move the player to the right.

Modify the player ship Update() function to contain the following code:

```
    if (RightKeyPressed())
    {
        MoveRight();
    }
```

The Update function should now look like the following:

```
void PlayerShip::Update()
{
    if (!m_bInitialised)
        return;

    if (LeftKeyPressed())
    {
        MoveLeft();
    }

    if (RightKeyPressed())
    {
        MoveRight();
    }
}
```

The code will now check if the left arrow key is being pressed and if it is, move the ship to the left. Then the code will check if the right arrow key is being pressed and if it is, move the ship to the right.

What would be the difference if we used an else if instead of two ifs?

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run then please ask a member of staff for help.**

You should now be able to move the player ship left and right using the arrow keys on the keyboard.

Task 4: Creating an army of enemies

You have made good progress getting the player ship to move left and right on the screen, but this doesn't make a very interesting game. Let's add some Space Invaders for us to shoot!

Step 1: Display the enemy army

There is already code that creates the enemy army so all we need to do is display them on the screen.

Open the Game.cpp file using the Solution Explorer as you have done previously.

Find the RenderGame() function in the Game.cpp file. Render is a fancy word for draw, so this function contains code to draw all of the things in our game. The code looks like the following:

Find Render Game function

```
void Game::RenderGame()
{
    m_PlayerShip.Render(m_pRenderer);

    RenderScore();
}
```

You can see that there is already code to draw the player ship and the score. These are the two things that we can already see in our game.

To draw our army of Space Invaders, add the following code:

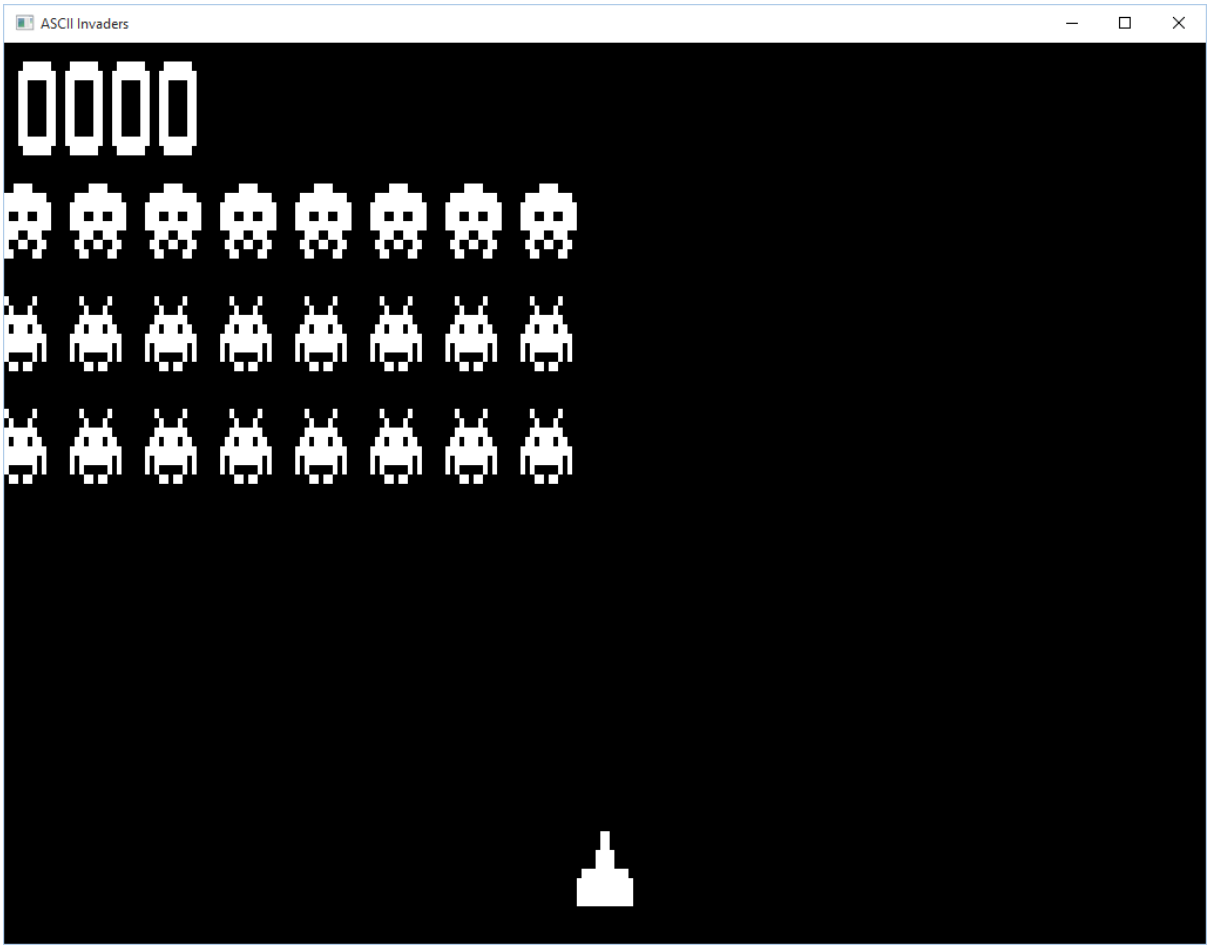
```
    m_EnemyArmy.Render(m_pRenderer);
```

The function should now look like the following:

```
void Game::RenderGame()
{
    m_PlayerShip.Render(m_pRenderer);

    m_EnemyArmy.Render(m_pRenderer);

    RenderScore();
}
```

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run, then please ask a member of staff for help.**

You should now see an army of Space Invaders in the game!



Task 5: Firing Missiles

Well done on getting the enemies to appear, but having enemies is no fun unless you can blast them to smithereens! Let's make the player fire missiles when you hit the space key.

Step 1: Checking if the space key has been pressed and firing

Previously when we checked if the left and right arrow keys were pressed, we called some nice functions that hid some code from us.

To check if a keyboard button is pressed or not, we can call a windows function that will tell us if the key has been pressed.

Find the UpdateGame() function in Game.cpp and add the following code:

```
if (GetKeyState(VK_SPACE) < 0)
{

}
```

What this code does is calls the GetKeyState() function provided by Microsoft. We tell it what key we are interested in, in this case the space key or VK_SPACE. Then we check if the value is less than 0, which means the key is pressed.

Inside of the curly braces, add the following code:

```
if (!m_Missile.Active())
{
    FireMissile();
}
```

What this code does is checks that there isn't already a missile active and if there isn't then call the FireMissile() function. This effectively limits the number of missiles a player can fire to one at a time.

The UpdateGame() function should now look like the following:

```
void Game::UpdateGame()
{
    if (GetKeyState(VK_SPACE) < 0)
    {
        if (!m_Missile.Active())
        {
            FireMissile();
        }
    }

    m_PlayerShip.Update();

}
```

Now before we compile and run the game again, we need to tell the game to draw the player missile too.

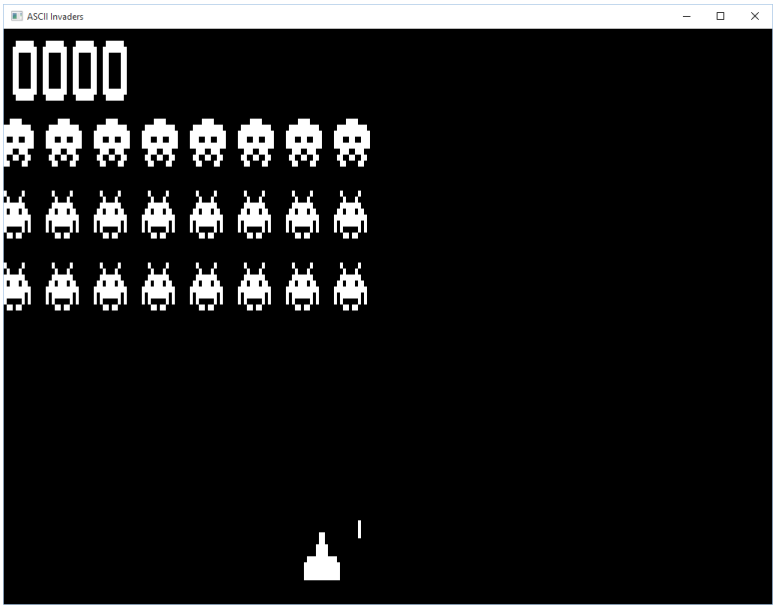
Find the RenderGame() function in the Game.cpp code file again and add the following code:

```
    m_Missile.Render(m_pRenderer);
```

The RenderGame() function should now look like the following:

```
void Game::RenderGame()
{
    m_PlayerShip.Render(m_pRenderer);

    m_EnemyArmy.Render(m_pRenderer);

    m_Missile.Render(m_pRenderer);

    RenderScore();
}
```

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run then please ask a member of staff for help.**



Run the game and press the Space key on the keyboard. You will notice that a missile is generated but it doesn't move anywhere! This is because we need to tell the game to update the missile.

Locate the UpdateGame() function in the Game.cpp code file and add the following code:

```
    UpdatePlayerMissiles();
```

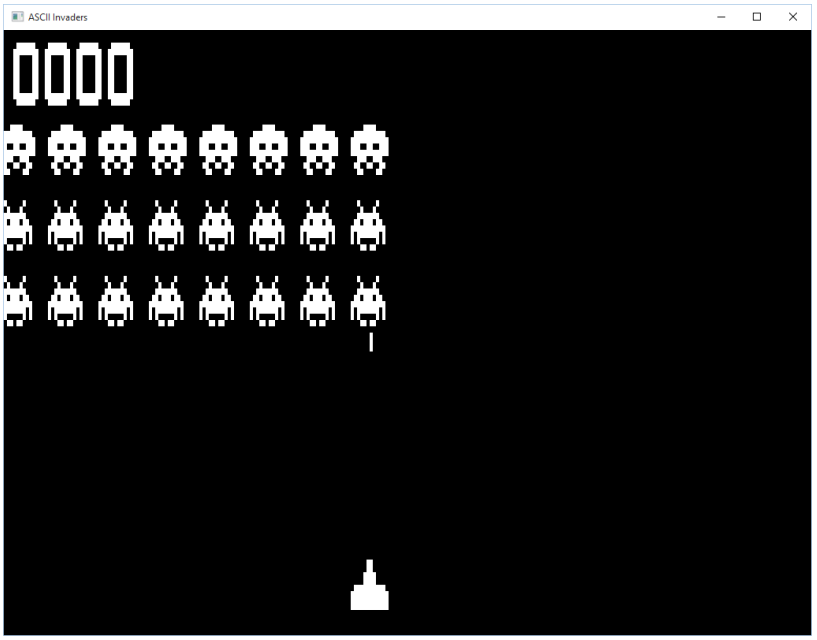
This will call the UpdatePlayerMissiles() function which will take care of all the player missiles we fire.

The UpdateGame() function should now look like the following:

```
void Game::UpdateGame()
{
    if (GetKeyState(VK_SPACE) < 0)
    {
        if (!m_Missile.Active())
        {
            FireMissile();
        }
    }

    m_PlayerShip.Update();

    UpdatePlayerMissiles();

}
```

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run then please ask a member of staff for help.**

Press the Space bar and watch as your missile fires up the screen!



Task 6: Making the enemies move

You've created an army of Space Invaders and can fire missiles at them, but they don't move so they're the game is still a bit boring. Let's make the army move like they did in the original Space Invaders game!

Step 1: Forward March

All of the movement code for the army has been taken care of for you. All you need to do is make sure that the Enemy Army Update() function is being called.

Locate the UpdateGame() function in the Game.cpp file and add the following code:

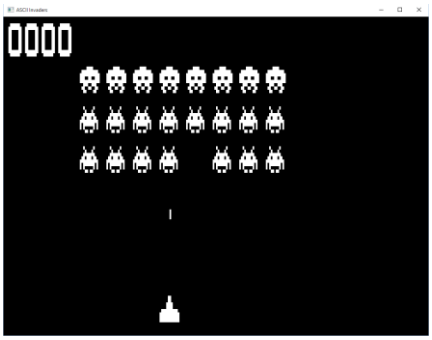
```
m_EnemyArmy.Update();
```

The UpdateGame() function should now look like the following:

```
void Game::UpdateGame()
{
    if(GetKeyState(VK_SPACE) < 0)
    {
        if(!m_Missile.Active())
        {
            FireMissile();
        }
    }

    m_PlayerShip.Update();

    UpdatePlayerMissiles();
    m_EnemyArmy.Update();

}
```

Compile and run the game as you have done previously. If the game does not run, then check that your code matches the code above exactly. **If after trying for a while and you still can't get you game to run, then please ask a member of staff for help.**

You should now have a moving army of Space Invaders to shoot at!



Task 7: Making the enemies fire back!

The game is getting a bit more fun now, but let's make it more interesting. We should make the Space Invader army fire back!

Step 1: Fire!

When you called the EnemyArmy Update() function in the previous task, you already took care of firing missiles from the enemies!

As with the player missile, we need to make sure that we update the enemy missiles and draw the enemy missiles to the screen.

Locate the UpdateGame() function in the Game.cpp file and add the following code:

```
UpdateEnemyMissiles();
```

This code will call the UpdateEnemyMissiles() function which will ensure that when the enemies fire missiles, they will shoot down the screen towards the player.

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run then please ask a member of staff for help.**

The missiles will not appear in the game yet as we have not drawn them to the screen. Let's do that now.

Locate the RenderGame() function in Game.cpp and add the following code:

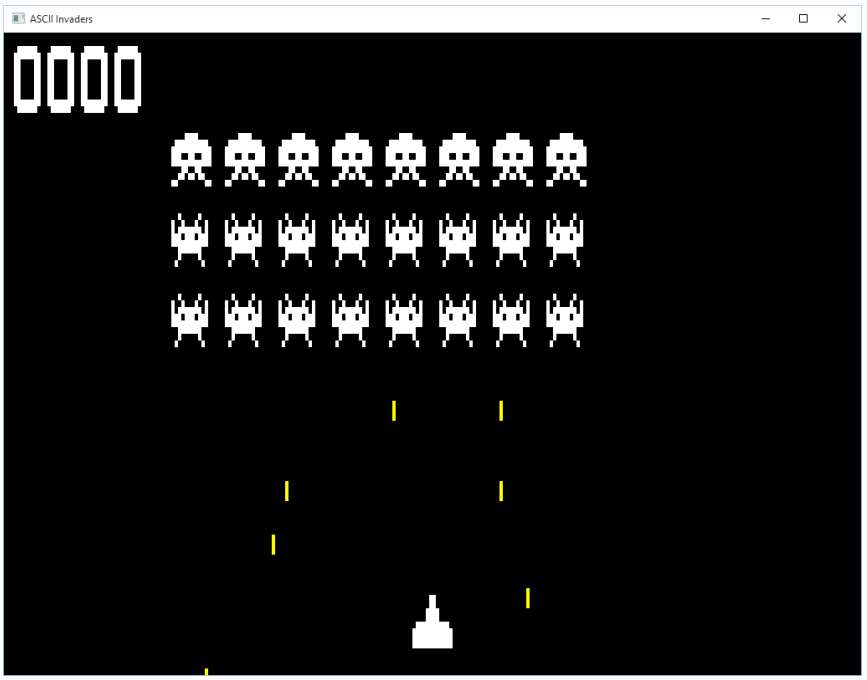
```
m_EnemyArmy.Render(m_pRenderer);
```

The RenderGame() function should now look like the following:

```
void Game::RenderGame()
{
    m_PlayerShip.Render(m_pRenderer);

    m_EnemyArmy.Render(m_pRenderer);

    m_Missile.Render(m_pRenderer);

    RenderEnemyMissiles();

    RenderScore();
}
```

Compile and run the game as you have done previously. If the game does not run, then check that your code matches the code above exactly.

The army of Space Invaders should now shoot back at you! Watch out because if you get hit then it'll be game over.



Task 8: Scoring points

We can now shoot the Space Invaders and they can shoot us, but no one is scoring any points. Let's add some code to make sure we get points when we destroy a Space Invader!

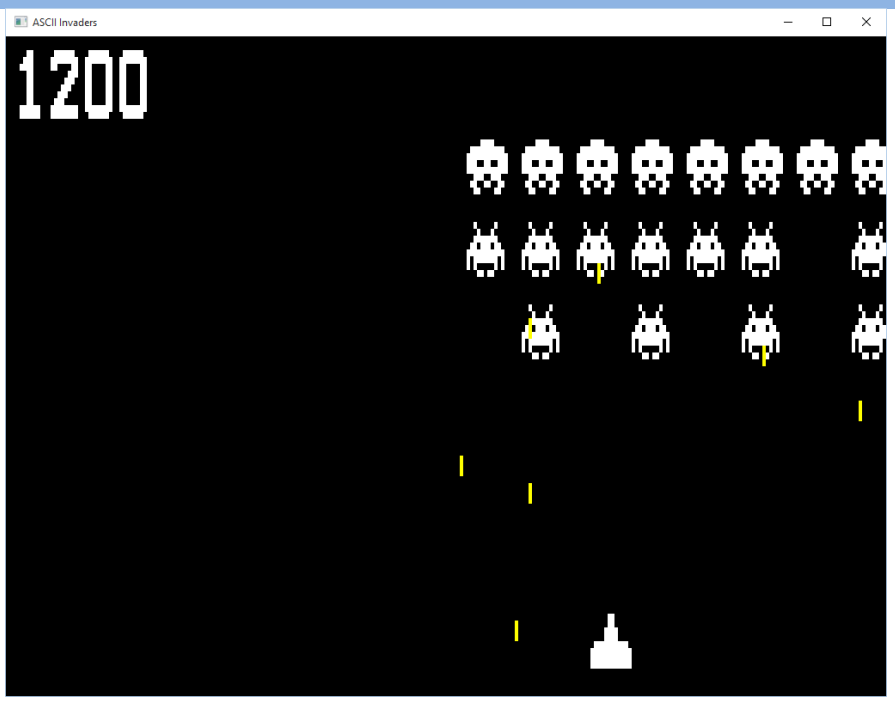
Step 1: Update the score

The score is already being calculated for us so all you need to do is to make sure that the score display is updated.

Locate the UpdateGame() function in Game.cpp and add the following code:

```
UpdateScoreDisplay();
```

Compile and run the game as you have done previously. If the game does not run, then check that your code matches the code above exactly. The score display should now update when you blast those Invaders.



Task 9: Winning the game

So you can blast the Space Invaders but nothing happens when they are all gone? We need to check if we have won the game and move to a win screen if we have.

Step 1: Winning!

We should win the game if we destroy all of the Space Invaders before they hit the bottom of the screen or shoot us. If we get shot or the Space Invaders hit the bottom of the screen then we lose.

We have a function that can do just that for us!

Locate UpdateGame() function in Game.cpp and add the following code:

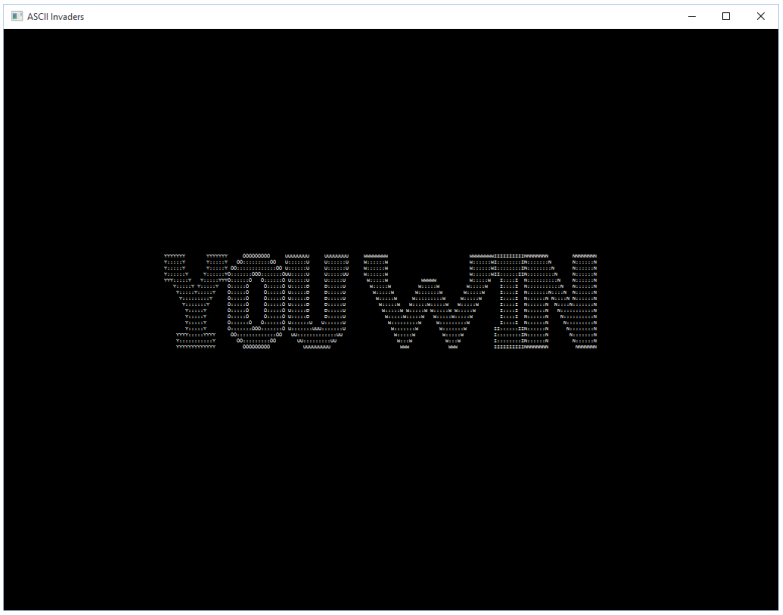
```
CheckWinConditions();
```

The UpdateGame() function should now look like the following:

```
void Game::UpdateGame()
{
    if(GetKeyState(VK_SPACE) < 0)
    {
        if(!m_Missile.Active())
        {
            FireMissile();
        }
    }

    m_PlayerShip.Update();
    UpdatePlayerMissiles();
    m_EnemyArmy.Update();
    UpdateEnemyMissiles();

    UpdateScoreDisplay();
    CheckWinConditions();
}
```

Compile and run the game as you have done previously. If the game does not run then check that your code matches the code above exactly. **If you can't get you game to run then please ask a member of staff for help.**

You should now be able to win and lose the game! Congratulations!



Advanced Task 1: Speed up player movement

Well done on getting this far! If there is some time left (and hopefully, there should be!), then there is an additional task for you. Can you figure out how to make the player move twice as fast when you press the arrow keys?

Step 1: ??

Hint: look at the code in the **PlayerShip.cpp** Code file where you made the player ship move

Congratulations on reaching the end of this recap session! Hopefully much of this is flooding back to you.

These tasks were designed as a quick reminder of the kind of programming tasks that you will face on this module – *we will be using a similar framework to build our game, but this time using the SFML framework, and our own Game Engine built on top of it* – we will be building upon the skills you learnt in Programming for Games and Mathematics and 2D Graphics in the first year.

What's Next?

With your now second-year C++ games developer hat on:

- **spend some time reviewing the type of code present in the files, the structure of the classes, and the way the game has been decomposed into functions – what do each of the functions encapsulate?**
- **how are they called, by what C++ concepts and syntax?**
- **Can you identify pointers? Can you identify standard variables – what about class instances?**

To help you with this, I advise using the Visual Studio Debugger to put some break points in the **UpdateGame** and **RenderGame** functions.

Step through some of the function calls.

Debugging is essential this year, so remembering how to use it will be essential – I will help you with this process during the module, but the basics are worth covering again by yourselves.